authclient 插件深度审查报告

> 版本：跟随主项目 | 类型：前端插件 | 代码量：~6,500 行 / 23 文件
>
> 审查日期：2026-03-27

综合评分

维度	评分	简评
功能性	8/10	7 种认证方式全覆盖
安全性	3/10	明文密码存储，客户端令牌验证
架构设计	5/10	巨型 Controller，职责过载
接口设计	6/10	Dio 拦截器链合理但耦合 GetX
错误处理	4/10	部分静默失败，错误类型不完整
可测试性	3/10	深度依赖 GetX，无法单元测试
总评	**5/10**	功能齐全但安全隐患严重

一、功能性分析

支持的认证方式

认证类型	实现	备注
游客登录	✅	匿名注册，自动登录
邮箱 + 密码	✅	注册、登录、重置
手机 + 密码	✅	注册、登录
邮箱验证码	✅	发送 + 验证
短信验证码	✅	腾讯云 SMS
Google OAuth	✅	第三方登录
Apple OAuth	✅	Sign In with Apple
微信 OAuth	✅	微信开放平台

双协议模式

```
enum AuthMode { json, protobuf }
```

支持 JSON REST 和 Protobuf 两种通信协议，通过 `AuthMode` 切换。

功能缺陷

缺陷	严重度	说明
无生物识别认证	🟡 中	不支持指纹/Face ID
无 2FA 支持	🟡 中	无 TOTP 等二次验证
无会话管理	🟡 中	无法查看/管理活跃会话
无密码修改 UI	🟡 中	只有重置密码，无已登录状态下改密

二、安全性批判

🔴 致命问题 1：密码明文持久化

```
class AuthStorage {
  static const _emailKey = 'auth_email';
  static const _passwordKey = 'auth_password';

  Future<void> saveCredentials(String email, String password) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_emailKey, email);
    await prefs.setString(_passwordKey, password);  // ← 明文！
  }

  Future<String?> getSavedPassword() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_passwordKey);  // ← 读取明文密码
  }
}
```

批判：

- SharedPreferences 在 Android 上是纯 XML 明文文件，root 设备可直接读取
- iOS上 UserDefaults 也非安全存储，不受 Keychain 保护
- 任何文件备份工具都能提取密码
- 这是 OWASP Mobile Top 10 的 M2（不安全数据存储）

改良方案：

```
// 使用 flutter_secure_storage（基于 Keychain/KeyStore）
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class SecureAuthStorage {
```

```
  final _storage = const FlutterSecureStorage();

  Future<void> saveCredentials(String email, String password) async {
    await _storage.write(key: 'email', value: email);
    await _storage.write(key: 'password', value: password);
  }
}
```

🔴 致命问题 2：自动保存凭据无用户同意

```
Future<void> login(String email, String password) async {
  final result = await _authApi.login(email, password);
  if (result.success) {
    // 登录成功后自动保存密码，未询问用户
    await _storage.saveCredentials(email, password);
    // ...
  }
}
```

- GDPR 和中国《个人信息保护法》要求明确告知用户
- 用户不知道密码被保存了
- 无"记住密码"选项提供给用户

🔴 致命问题 3：客户端令牌验证

```
bool get isTokenValid {
  if (_accessToken == null || _tokenExpiry == null) return false;
  return DateTime.now().isBefore(_tokenExpiry!);
}
```

批判：

- 使用设备本地时间判断 Token 是否过期
- 用户修改设备时间 → Token "永不过期"
- 安全的做法是让服务器拒绝过期 Token（返回 401），客户端捕获后刷新

🟠 高风险问题：Token 刷新竞态

```
class AuthRefreshInterceptor extends Interceptor {
  @override
  void onError(DioException err, ErrorInterceptorHandler handler) async {
    if (err.response?.statusCode == 401) {
      final newToken = await _authController.refreshToken();
      // 用新 Token 重试原请求
      final response = await _dio.fetch(err.requestOptions);
      handler.resolve(response);
    }
  }
}
```

- 10 个请求同时收到 401 → 10 个并发刷新请求
- 第 1 个刷新成功后，旧 refresh token 已失效
- 第 2-10 个刷新请求全部失败 → 用户被登出

改良方案：

```
class AuthRefreshInterceptor extends QueuedInterceptor {
  // QueuedInterceptor 确保同一时间只有一个请求在处理
  // 后续请求排队等待
}
```

三、架构设计批判

问题 1：巨型 AuthController

```
class AuthController extends GetxController {
  // 750+ 行！

  // 状态管理
  final Rx<AuthState> state = AuthState.unauthenticated.obs;
  final Rx<User?> currentUser = Rx<User?>(null);

  // 登录方法（7 种认证方式 × 各自逻辑）
  Future<void> loginWithEmail(...) { ... }
  Future<void> loginWithPhone(...) { ... }
  Future<void> loginWithGoogle(...) { ... }
  Future<void> loginWithApple(...) { ... }
  Future<void> loginWithWechat(...) { ... }
  Future<void> loginAsGuest() { ... }
  Future<void> loginWithCode(...) { ... }

  // Token 管理
  Future<void> refreshToken() { ... }
  Future<void> saveTokens(...) { ... }

  // 用户信息
  Future<void> updateProfile(...) { ... }
  Future<void> changeAvatar(...) { ... }

  // 注册
  Future<void> register(...) { ... }
  Future<void> sendVerificationCode(...) { ... }

  // 登出
  Future<void> logout() { ... }
}
```

批判：

- 750 行 Controller 承担认证、状态管理、Token 管理、用户资料管理所有职责

- 添加新的认证方式需要修改这个巨型类
- 违反 SRP 和 OCP

改良方案：

```
// 拆分为策略模式
abstract class AuthStrategy {
  Future<AuthResult> authenticate(Map<String, dynamic> params);
}

class EmailAuthStrategy implements AuthStrategy { ... }
class GoogleAuthStrategy implements AuthStrategy { ... }
class AppleAuthStrategy implements AuthStrategy { ... }

class AuthController extends GetxController {
  final Map<AuthType, AuthStrategy> _strategies;

  Future<void> login(AuthType type, Map<String, dynamic> params) {
    return _strategies[type]!.authenticate(params);
  }
}
```

问题 2：深度绑定 GetX

```
class AuthController extends GetxController {
  final email = ''.obs;           // GetX Observable
  final password = ''.obs;        // GetX Observable
  final isLoading = false.obs;    // GetX Observable

  @override
  void onInit() {
    super.onInit();
    ever(state, _onStateChanged);  // GetX Reactive
    // ...
  }
}
```

- 核心认证逻辑与 GetX 框架深度耦合
- 如果需要迁移到 Riverpod/Bloc，整个认证层需要重写
- 业务逻辑应抽取到框架无关的 UseCase 层

问题 3：Dio 拦截器链顺序不透明

```
_dio.interceptors.addAll([
  TokenInterceptor(),          // 1. 附加 Token
  AuthRefreshInterceptor(),    // 2. 401 时刷新
  LoggerInterceptor(),         // 3. 日志记录
  RetryInterceptor(),          // 4. 失败重试
]);
```

拦截器执行顺序隐式依赖 `addAll` 的数组顺序。如果 `RetryInterceptor` 放到 `AuthRefreshInterceptor` 前面，重试不会携带新 Token。缺少文档说明这个顺序为什么是正确的。

四、接口设计批判

问题 1：Proto 响应解析不完整

```
Future<AuthResult> loginProto(String email, String password) async {
  final response = await _dio.post('/api/auth/login',
    data: LoginRequest(email: email, password: password).writeToBuffer(),
  );

  // 有时解析 proto，有时手动构造
  if (response.data is List<int>) {
    return AuthResult.fromBuffer(response.data);
  } else {
    // 降级为 JSON 解析？还是抛异常？不确定
    return AuthResult.fromJson(response.data);
  }
}
```

Proto 和 JSON 两套解析逻辑在同一方面混用，容易出错。

问题 2：AuthResult 返回值不统一

```
// 有时返回 AuthResult
Future<AuthResult> login(...);

// 有时返回 bool
Future<bool> sendCode(...);

// 有时返回 void
Future<void> logout();

// 有时返回 Map
Future<Map<String, dynamic>> getUserInfo();
```

四种不同的返回值模式，调用者需要每个方法单独处理。

改良方案：

```
class Result<T> {
  final T? data;
  final String? error;
  final bool success;
}

Future<Result<AuthResult>> login(...);
Future<Result<void>> sendCode(...);
Future<Result<void>> logout(...);
```

```
Future<Result<UserInfo>> getUserInfo(...);
```

五、改良建议总结

优先级	改良项	预估工作量
P0	密码改用 flutter*secure*storage	2h
P0	保存凭据前征求用户同意	1h
P0	移除客户端 Token 过期判断，依赖 401 响应	2h
P0	AuthRefreshInterceptor 改用 QueuedInterceptor	2h
P1	拆分 AuthController 为策略模式	8h
P1	认证逻辑从 GetX 解耦到 UseCase 层	6h
P1	统一返回值为 Result<T>	4h
P2	Proto/JSON 响应处理分离	3h
P2	拦截器顺序文档化	1h
P3	支持生物识别认证	8h
P3	支持 2FA (TOTP)	8h